DISEÑO DE SOFTWARE · PROYECTO G8 · EMPRESA DE VIGILANCIA

VigiControl

Minuta digital de vigilancia: turnos, llegadas y novedades

Jairo Andrés Zubieta Bermúdez
Ingeniería de Sistemas · Universidad del Tolima

Metodología ágil · Software libre
Python · Flask · SQLite · PlantUML

EL PROBLEMA

La minuta todavía es un libro de papel

En cada puesto, el vigilante anota a mano su llegada y lo que pasa en el turno. Esa información queda en el puesto y el supervisor se entera tarde o nunca.

¿Quién llegó?

No se sabe en tiempo real quién llegó a su puesto ni quién llegó tarde.

¿Qué pasó?

Las novedades quedan en el libro del puesto y nadie las revisa a tiempo.

¿Quién lo arregla?

Las fallas técnicas se avisan por teléfono y no queda registro de la solución.

LA SOLUCIÓN

VigiControl: una minuta en el celular, para tres roles

Vigilante

Marca llegada y salida, y registra las novedades del turno.

Técnico

Atiende las fallas asignadas: marca su llegada y cierra con la solución.

Supervisor

Programa turnos, asigna las fallas y ve todo en un panel.

1 Vigilante marca llegada → 2 Reporta la falla → 3 Supervisor asigna → 4 Técnico llega y la cierra

METODOLOGÍA ÁGIL

Scrum adaptado: tres sprints de construcción

Sprint	Objetivo	Historias	Puntos
0	Análisis y diseño UML	—	—
1	Acceso y datos básicos	HU01 – HU04	10
2	El turno del vigilante	HU05 – HU07	11
3	Novedades técnicas y panel	HU08 – HU10	13
Backlog	Fotos y GPS	HU11	8

Artefactos: product backlog, sprint backlog, tablero Kanban y estimación en puntos de historia. **Hecho** = cumple sus criterios de aceptación y funciona en el prototipo.

REQUISITOS FUNCIONALES

El prototipo cubre 9 de 10 requisitos

Implementados

- RF01 Iniciar sesión por rol
- RF02 Gestionar puestos
- RF03 Gestionar personal
- RF04 Programar turnos
- RF05 Marcar llegada y salida
- RF06 Registrar novedades
- RF07 Minuta del puesto
- RF08 Asignar y cerrar fallas
- RF09 Panel del supervisor

90 % de cobertura (mínimo: 50 %)

Pendiente

RF10 Foto en la novedad y GPS al marcar

Reglas clave: 10 min de tolerancia para llegar · sin turnos cruzados · novedades solo con la llegada marcada

HISTORIAS DE USUARIO

Once historias, priorizadas por valor

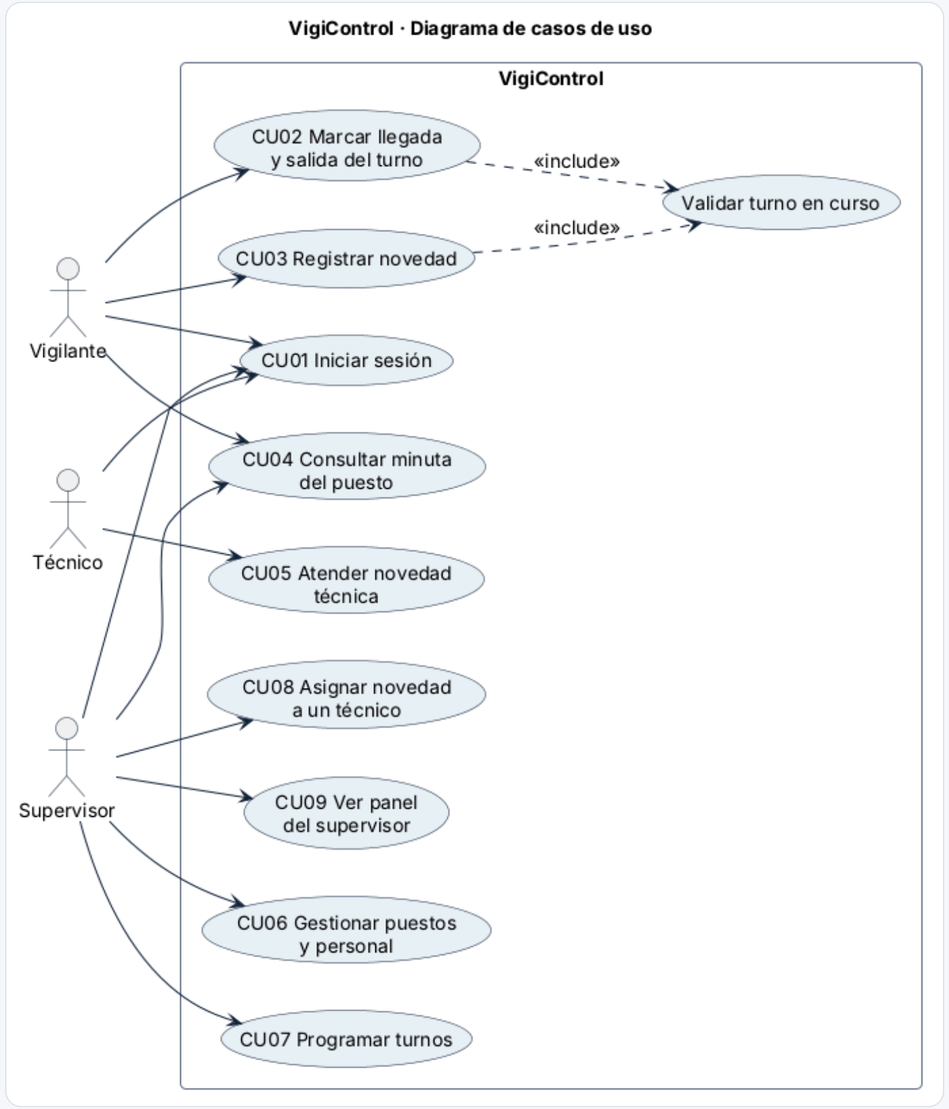
HU	Como…	quiero…	Pts	Sprint
HU01	Usuario	iniciar sesión y ver solo las opciones de mi rol	2	1
HU02	Supervisor	registrar los puestos de vigilancia	2	1
HU03	Supervisor	registrar al personal y darle acceso	3	1
HU04	Supervisor	programar turnos sin cruces de horario	3	1
HU05	Vigilante	marcar mi llegada y salida del turno	5	2
HU06	Vigilante	registrar las novedades del turno	3	2
HU07	Supervisor	ver la minuta de cada puesto	3	2
HU08	Supervisor	asignar las fallas técnicas a un técnico	3	3
HU09	Técnico	marcar mi llegada y cerrar la falla con la solución	5	3
HU10	Supervisor	ver un panel con el estado de los turnos	5	3
HU11	Vigilante	adjuntar foto y ubicación a la novedad	8	Backlog

UML · CASOS DE USO

Tres actores, nueve casos de uso

- **Vigilante:** marca llegada y salida, registra novedades y consulta la minuta.
- **Técnico:** atiende las novedades técnicas.
- **Supervisor:** gestiona puestos, personal y turnos, asigna novedades y ve el panel.
- Marcar y registrar **incluyen** «validar turno en curso».

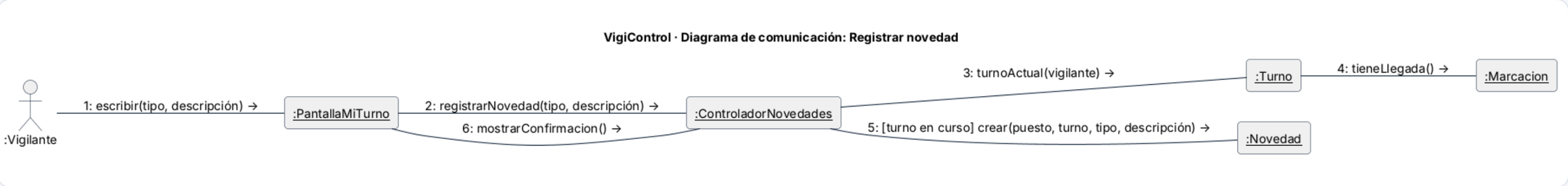
Herramienta CASE: PlantUML · uml/01-casos-de-uso.puml



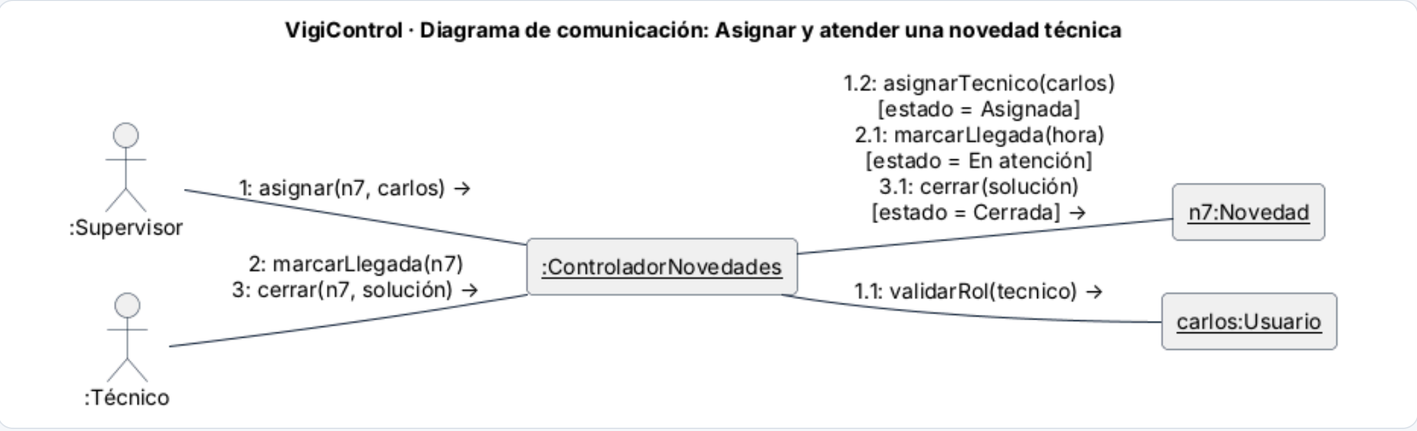
UML · DIAGRAMAS DE COMUNICACIÓN

Cómo colaboran los objetos

1 · Registrar novedad: solo si el turno está en curso



2 · Asignar y atender una novedad técnica



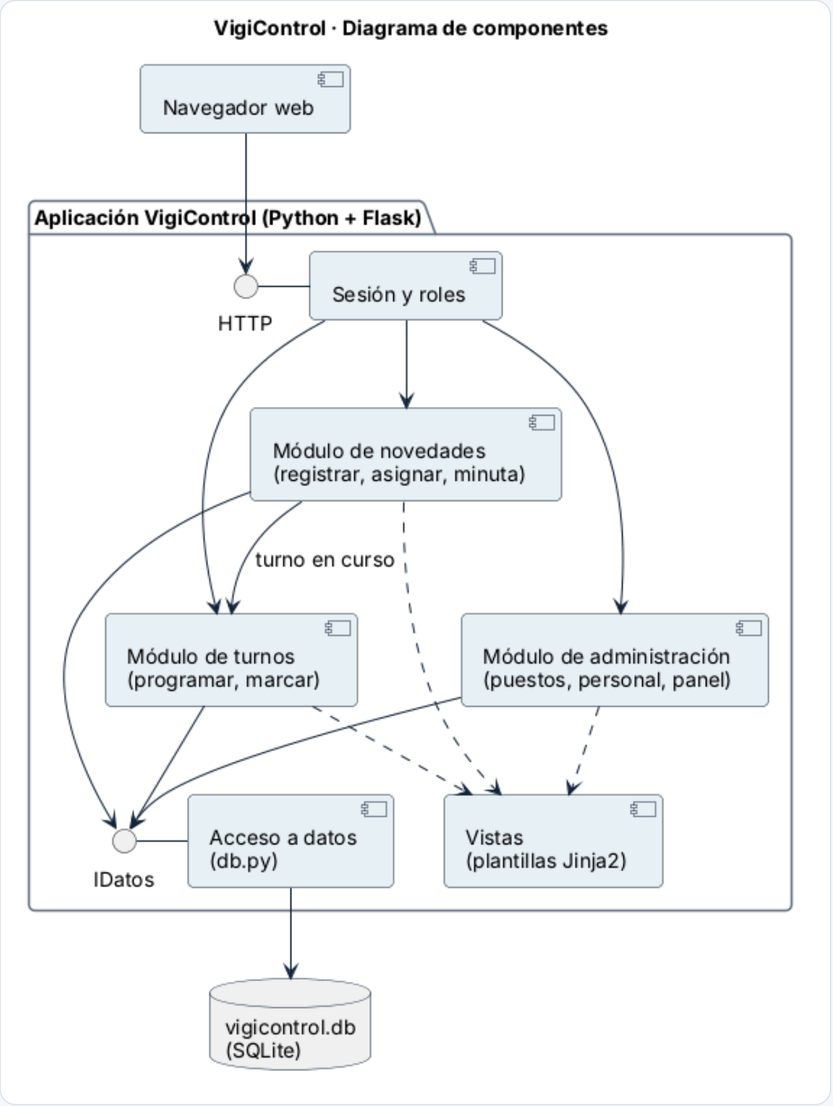
Los números indican el orden de los mensajes. La novedad cambia de estado: **Asignada → En atención → Cerrada**.

UML · COMPONENTES

Una aplicación, tres módulos

- **Sesión y roles** recibe cada petición y revisa quién es el usuario.
- **Turnos**, **novedades** y **administración** son los módulos de negocio.
- Novedades usa a Turnos para saber si hay un turno en curso.
- Solo el **acceso a datos** habla con SQLite.

uml/04-componentes.puml

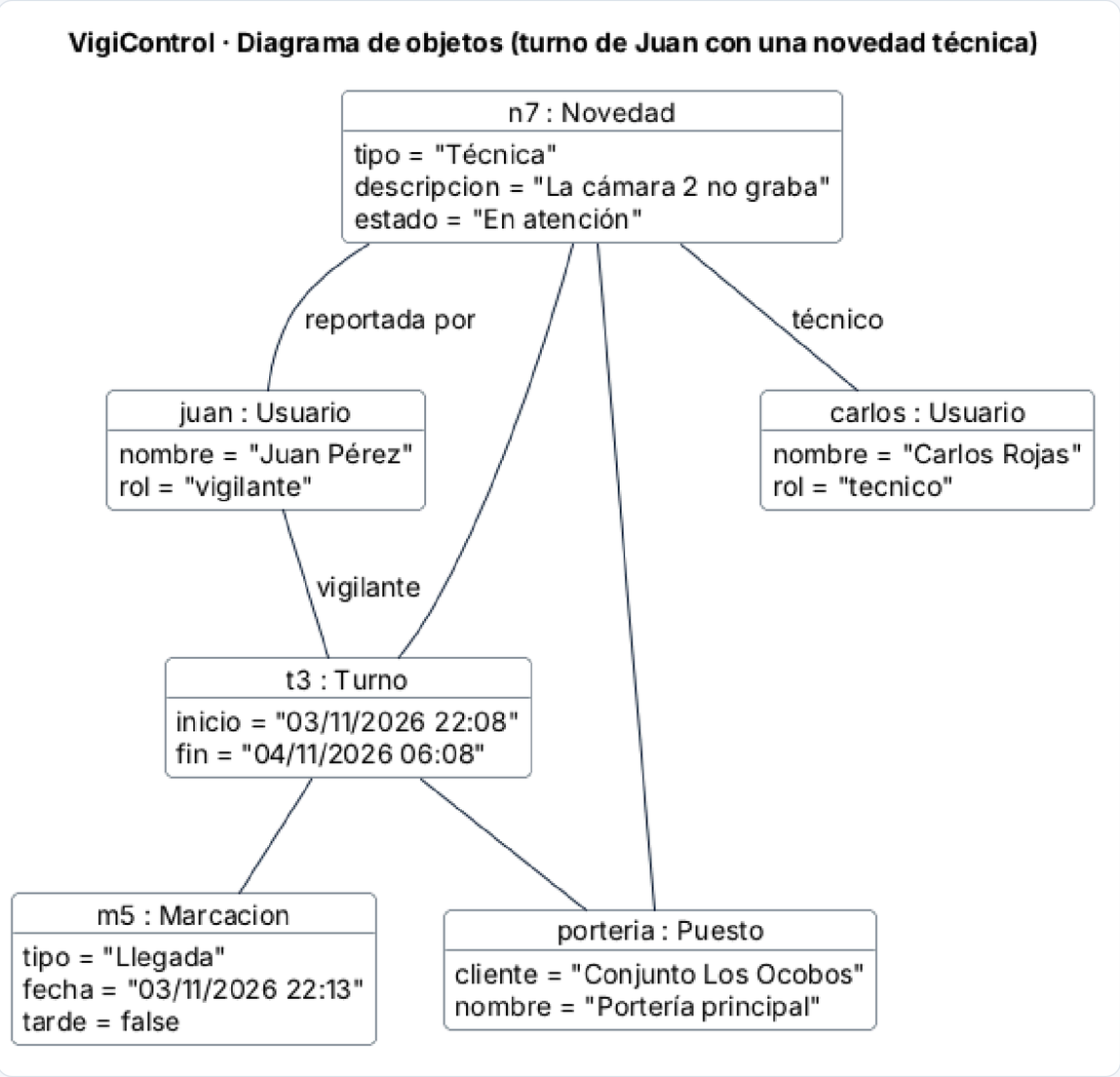


UML · OBJETOS

Una foto del sistema en un momento dado

- **juan** tiene el turno **t3** en la portería.
- La marcación **m5** registra su llegada a tiempo.
- Juan reportó la novedad **n7**: «la cámara 2 no graba».
- El técnico **carlos** la está atendiendo.

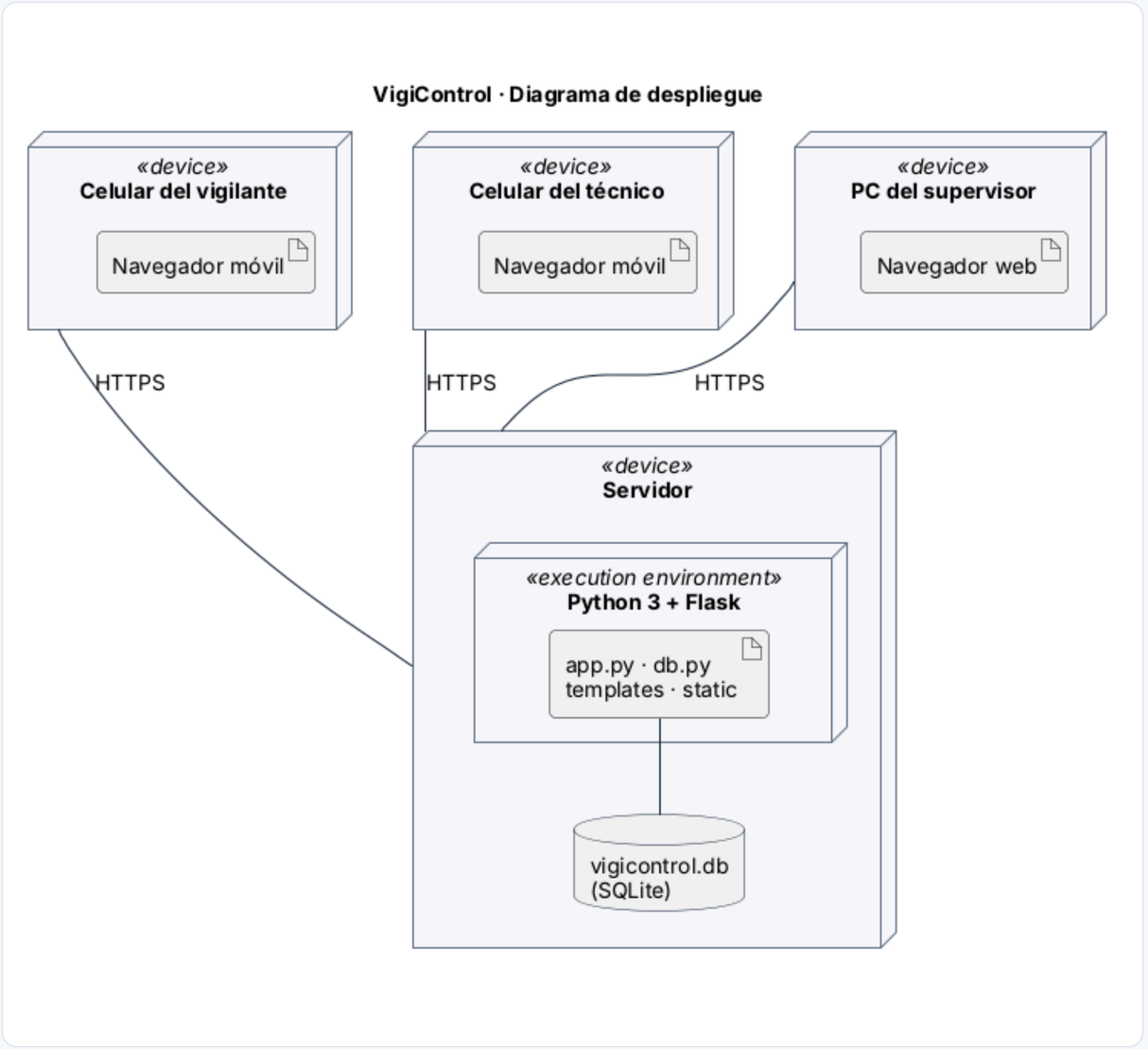
uml/05-objetos.puml · el diagrama de clases de apoyo está en uml/07-clases.puml



UML · DESPLIEGUE

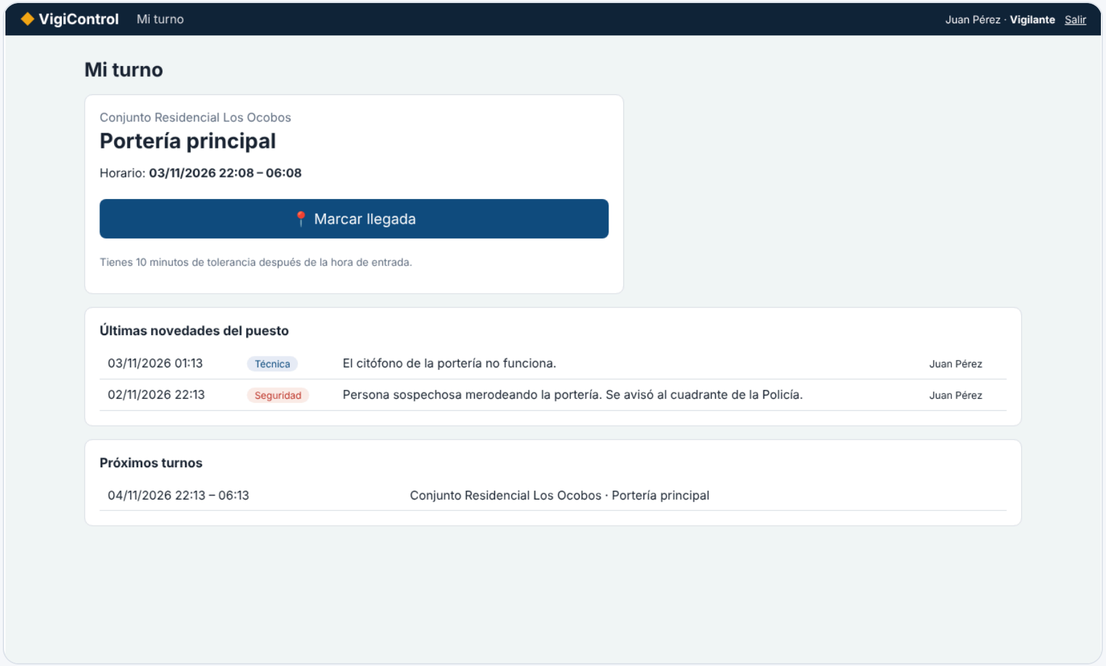
Celulares, un PC y un servidor

- Vigilantes y técnicos usan el **navegador del celular**.
- El supervisor usa el **navegador del PC**.
- Todos se conectan por **HTTPS** al servidor, donde corren Python y Flask.
- Los datos viven en un archivo **SQLite** en el servidor.

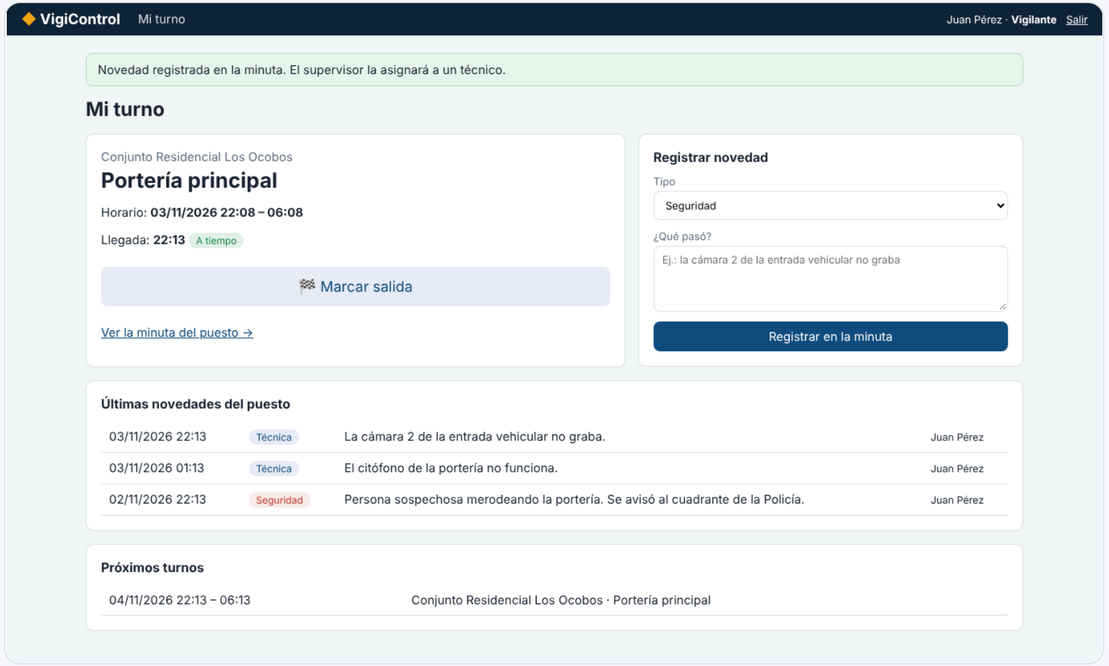


PROTOTIPO · VIGILANTE

Marcar llegada y registrar la novedad



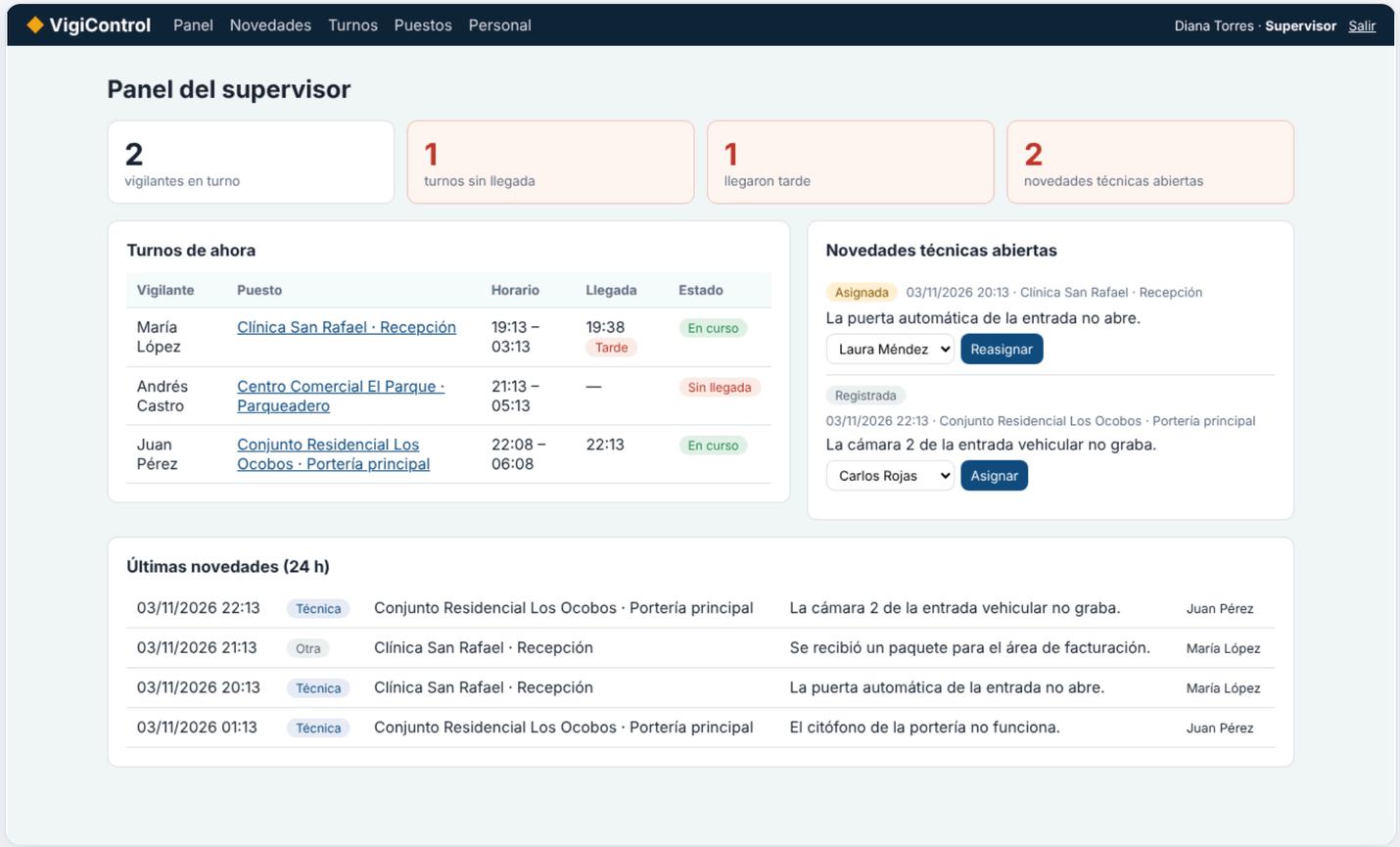
1. Juan ve su turno y marca la llegada. Si pasa la tolerancia de 10 min, queda «Tarde».



2. Con la llegada marcada, registra la novedad técnica en la minuta.

PROTOTIPO · SUPERVISOR

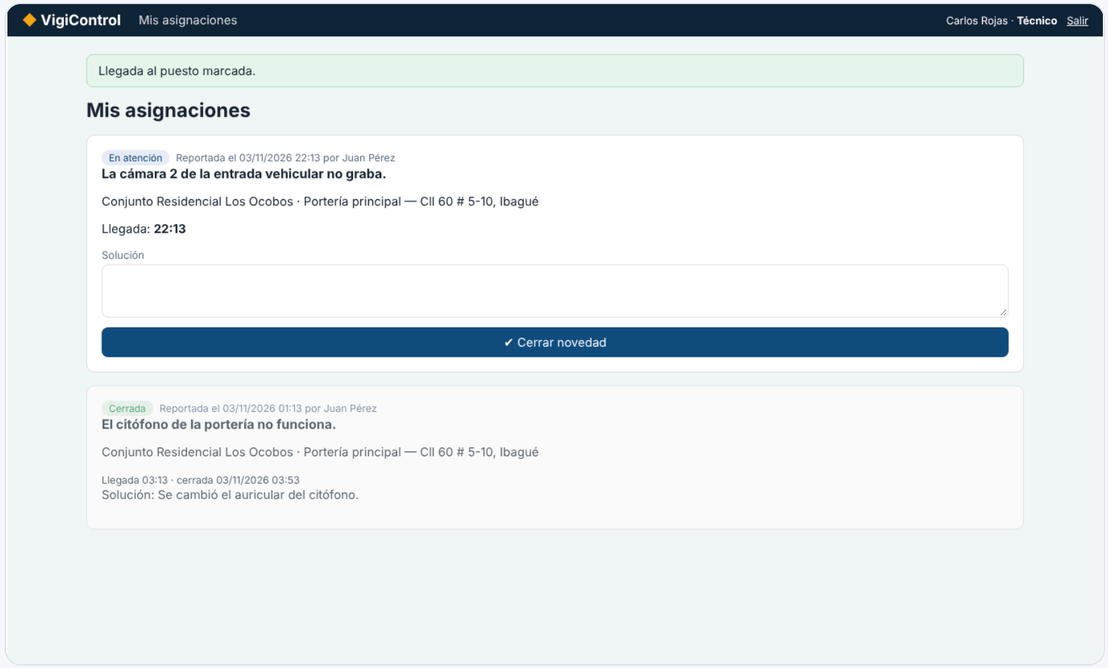
Todo el servicio en un panel



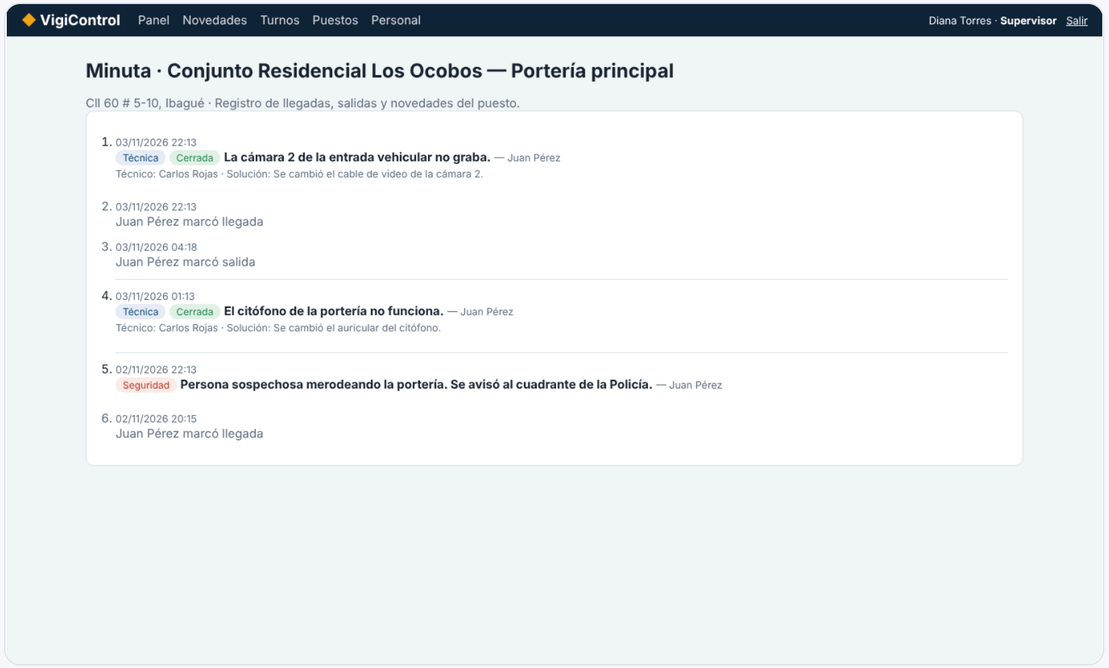
- Cuántos vigilantes están en turno.
- Quién **no ha llegado** y quién llegó **tarde**, en rojo y con texto.
- Las novedades técnicas abiertas, con el botón para **asignar** al técnico.
- Las novedades de las últimas 24 horas.

PROTOTIPO · TÉCNICO Y MINUTA

La falla se cierra y queda en la minuta



3. Carlos marca su llegada al puesto y cierra la novedad con la solución.



4. La minuta del puesto muestra todo en orden: llegadas, novedades y soluciones.

DEMOSTRACIÓN

Dos formas de probarlo

En el navegador

En la página del curso, entrada **VigiControl** → demo en vivo. Corre el mismo código Python en el navegador, sin instalar nada.

En el computador

```
pip install -r requirements.txt
python db.py
python app.py
```

Usuario	Contraseña	Rol
juan	vigilante123	Vigilante
supervisor	supervisor123	Supervisor
carlos	tecnico123	Técnico

Hecho con software libre: Python 3 · Flask · SQLite · PlantUML · VS Code · Git

CONCLUSIONES

De la minuta de papel a la minuta digital

Diseño primero

Las historias de usuario y los diagramas UML definieron qué construir y en qué orden.

90 % construido

9 de 10 requisitos funcionando: llegadas, novedades, asignación a técnicos y panel.

Siguiente versión

Fotos en las novedades y GPS al marcar, para tener evidencia de cada registro.

¡Gracias! ¿Preguntas?